

R102TP4

I)

1)

Utilisation scapy :

Sur la VM Kali :

```
[~] $ mkdir R102TP4
```

```
[~] $ cd R102TP4
```

```
[~/ R102TP4 ] $ sudo python3
```

2)

```
>>> from scapy . all import *
```

```
>>> p = Ether () / IP () / ICMP () → Création d'un paquet
```

```
>>> p
```

Réponse obtenue :

```
<Ether type=IPv4 |<IP frag=0 proto=icmp |<ICMP |>>>
```

Cette réponse représente l'encapsulation d'un paquet IP dont la charge utile est ICMP.

Le protocole encapsulé dans Ethernet est IP version 4, le paquet n'a pas été fragmenté et encapsule ICMP.

Méthodo : Pour créer des paquets vous allez ainsi indiquer ce que vous voulez comme encapsulation en séparant chaque protocole avec le symbole /

3)

p.summary() affiche :

```
'Ether / IP / ICMP 127.0.0.1 > 127.0.0.1 echo-request 0'
```

Ce qui correspond à la même chose que la commande p mais avec spécification du type d'ICMP (echo-request (ici), etc.) et les adresses IP source et destination.

p.show renvoie :

```
'Ether / IP / ICMP 127.0.0.1 > 127.0.0.1 echo-request 0'
```

ls(p) :

renvoie toutes les informations des trames et en-têtes créés.

4)

Les valeurs par défaut sont « None » pour l'adresse MAC (dest : ff:ff:ff:ff:ff:ff et src : 00:00:00:00:00:00)

Les valeurs par défaut sont « None » pour l'adresse IP c-à-d l'adresse de loopback (dest : 127.0.0.1 et src : 127.0.0.1)

Le TTL par défaut est 64

Le type de message ICMP par défaut est 8 (echo-request)

5)

Obtenir un hexdump du paquet :

```
>>> hexdump ( p )
```

6) Examiner le paquet sous wireshark :

```
wrpcap (" p . pcap " , p )
```

L'opération inverse, lire un pcap depuis Scapy, est aussi possible avec la fonction rdpcap.
Pour ouvrir le fichier sous Wireshark, il faut l'ouvrir depuis l'explorateur de fichier et cela ouvrira instantanément wireshark.

7)

Afin de mettre tous les paquets d'un fichier wireshark dans une variable :
`p = rdpcap (" icmp . pcap ")`

Pour charger le fichier du moodle entrer le proxy dans firefox sur la VM : Firefox, Settings, Networks Settings (tout en bas), Settings, cocher automatic proxy configuration URL et entrer `http://iut-rt.edu.ua/proxy.pac`

Puis lorsque l'on tape p, cela renvoie :
`<icmp.pcap: TCP:0 UDP:4 ICMP:2 Other:2>`
Qui signifie :

0 paquets TCP

4 paquets UDP (tous des DNS)

2 paquets ICMP (ping)

2 paquets "Other" → ARP

Décodage par paquet :
`p.show()`

8)

Pour examiner une trame depuis scapy, voici la commande :
`p[numéro de la trame (commence à zéro)].show()`

9)

Les slicings fonctionnent également sous la forme : `p[0:3].show()`

10)

Pour construire un ping vers la rtbox avec scapy :

```
p = Ether () / IP ( dst = '192.31.25.1 ' ) / ICMP ()
```

Quand on fait `p.show`, on obtient pareil que précédemment mais avec les bonnes IP renseignées.

11)

Vous pouvez accéder (et modifier) les champs avec par exemple :

```
>>> p [ IP ]. dst
>>> p [ IP ]. src
>>> p [ Ether ]. dst
>>> p [ Ether ]. src
>>> p [ ICMP ]. type
```

II)

1) Envoyer des paquets en fonction de la couche où intervient le premier protocole de votre encapsulation :

```
send() = IP/ICMP
sendp() = Ether()/IP()/ICMP()
```

```

send (p )
p = IP ( dst = '192.31.25.1 ' ) / ICMP ()

p = Ether () / IP ( dst = '192.31.25.1 ' ) / ICMP ()
srp1 (p )

```

2)

Obtenir la réponse au ping :

```

>>> p = Ether () / IP ( dst = '192.31.25.1 ' ) / ICMP ()
>>> srp1 (p )

```

Puis

```

>>> p = IP ( dst = '192.31.25.1 ' ) / ICMP ()
>>> sr1 ( p)

```

Puis

```

>>> p = Ether () / IP ( dst = '192.31.25.1 ' ) / ICMP ()
>>> rep = srp1 ( p) → après p on peut mettre virgule timeout=valeur pour imposer un timeout
>>> rep . summary ()
>>> rep . Show ()

```

3)

Si l'adresse n'est pas atteignable, quand on fait print(rep) on obtient None.

III)

```
#!/usr/bin/env python3
```

```

import argparse
import logging
from scapy.all import *

```

```

# Supprimer les messages inutiles de Scapy
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)

```

```

parser = argparse.ArgumentParser(description="Scan ping")
parser.add_argument(
    "max",
    type=int,
    help="Valeur du dernier octet max des adresses IP à scanner"
)

```

```
args = parser.parse_args()
```

```

# Mode silencieux (quiet)
conf.verb = 0

```

```

for i in range(1, args.max + 1):
    ip = "192.31.25." + str(i)
    p = IP(dst=ip) / ICMP()

```

```
    rep = sr1(p, timeout=5)
```

```

    if rep is not None:
        print(f"{ip} répond (host up)")
    else:
        print(f"{ip} ne répond pas (host down)")

```

IV)

1)
ETH | ARP

2)
Trame ARP :

```
p = Ether ( dst = ' ff : ff : ff : ff : ff : ff ' ) / ARP ( pdst  
= '192.31.25.1 ' )
```

pdst = packet destination

V)

1) ETH | IP | UDP | DNS

2)
Etape 1 : interroger le serveur DNS pour obtenir son adresse IP :
nslookup rt - bethune . univ - artois . fr
Serveur DNS :
p = IP (dst = ' @IP_serveur_DNS ') / UDP () / DNS (qd = DNSQR (qname = ' rt - bethune . univ - artois . Fr '))

VI)

Scan de ports : TCP :
p = IP (dst = ' @IP_passerelle_RTBox2 ') / TCP (dport =22 ,
flags = 'S ')

Testez avec d'autres ports : 20, 23 et 80.

20 = RA

23 = RA

80 = SA

```
#!/usr/bin/env python3
```

```
import argparse  
import logging  
from scapy.all import *
```

```
# Supprimer les messages inutiles de Scapy  
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
```

```
parser = argparse.ArgumentParser(description="Scan ports")  
parser.add_argument("ip", type=str, help="Adresse IP à scanner")  
args = parser.parse_args()
```

```
PORTS_A_SCANNER = [20, 22, 23, 80]
```

```
# Mode silencieux  
conf.verb = 0
```

```
for i in PORTS_A_SCANNER:  
    # Création du paquet TCP SYN  
    p = IP(dst=args.ip) / TCP(dport=i, flags="S")
```

```
# Envoi du paquet
rep = sr1(p, timeout=2)

# Test : réponse non vide + TCP + flags SYN/ACK
if rep is not None and rep.haslayer(TCP) and rep[TCP].flags == 0x12:
    print(f"Port {i} ouvert sur {args.ip}")
```